Case Study: TCP vs UDP for Real-Time Video Calls over Lossy Wi-Fi

Computer Networks (BACSE203) – Transport Layer Case Study

Team members: [Your Name] [Register No.] | Date: 02/11/2026

Code and data: https://github.com/abin3007/CN_CaseStudy_TCP_vs_UDP

1. Problem statement

Since online classes became normal, students and teachers use Zoom, Google Meet, Microsoft Teams and WhatsApp calls every day, very often over a crowded campus or hostel Wi-Fi network or over weak 4G. The common complaint is not that the connection is "slow": a 400 kbit/s video call needs far less bandwidth than these networks offer. The complaint is that the video freezes, the voice cuts out and people talk over each other. The real cause is packet loss, delay and jitter on the wireless link, and how the transport layer reacts to them.

TCP and UDP react in opposite ways. TCP guarantees that every byte arrives and in order, by retransmitting lost segments and slowing down when it sees loss. UDP gives no guarantee at all. For a file download TCP is clearly right. For a live call, a packet that arrives after it should have been played is as useless as a lost one. This case study asks the question that a network or application designer has to answer: which transport gives the best live call on a lossy Wi-Fi network, and can we do better than both?

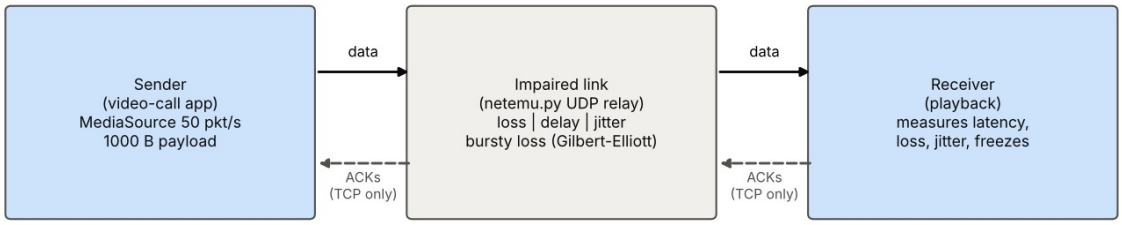
Objective and tangible outcome. We built a network emulator and three working transports in Python, sent the same video-call stream through six realistic network conditions and a loss sweep, and produced a protocol comparison with performance tables, per-packet latency graphs and a TCP congestion-window trace (Sections 4 and 5).

2. Background

- **TCP.** Connection-oriented and reliable. Each segment carries a sequence number; the receiver returns a cumulative ACK. A lost segment is retransmitted after three duplicate ACKs (fast retransmit) or after the retransmission timeout (RTO). The RTO is computed from smoothed RTT and RTT variance (RFC 6298): RTO = SRTT + 4·RTTVAR, minimum 200 ms in Linux. Each timeout doubles the RTO (exponential back-off). Data is given to the application strictly in order, so a single missing segment holds back every later segment that has already arrived. This is head-of-line (HOL) blocking. Congestion control (slow start, congestion avoidance, fast recovery; RFC 5681) halves the window on loss and resets it to 1 segment on a timeout.
- **UDP.** Connectionless, no ACKs, no retransmission, no ordering, 8-byte header. Real-time media protocols (RTP, WebRTC, VoIP) run on UDP and let the application decide what to do about loss: usually play what arrives and hide small gaps (packet loss concealment).
- **Forward Error Correction (FEC).** The sender adds redundant data so that the receiver can rebuild lost packets without asking for a retransmission. The simplest form, used here, sends one XOR parity packet after every K = 4 data packets; any single loss in a group of 4 can be recovered. WebRTC uses the same idea (ULPFEC/FlexFEC, RFC 5109 / RFC 8627).
- **Deadline.** ITU-T G.114 recommends that one-way mouth-to-ear delay stays below 150 ms for good interactive conversation. We use 150 ms as the playout deadline: a packet that arrives later than 150 ms after it was generated cannot be played.
- **TCP throughput under loss.** For TCP the Mathis et al. model gives the maximum steady throughput as roughly (MSS/RTT)·1.22/√p, where p is the loss rate. If this is below the bit rate of the call, TCP cannot keep up and data piles up in the send buffer.

3. Setup and method

All three transports exchange real UDP datagrams over the loopback interface of one computer (Python 3.11, Linux; the code also runs on Windows and macOS). Between sender and receiver sits our own impairment emulator, netemu.py, which acts like a lossy Wi-Fi access point or router (Graph 1). We wrote it instead of using Linux tc netem so that the experiment needs no administrator rights and works on any OS.



Graph 1. Experimental topology. Data and ACKs both pass through the impaired link.

- **Impaired link (netemu.py).** Drops packets with probability p (Bernoulli), or in bursts using a two-state Gilbert-Elliott model (average loss p, mean burst length 3 packets) to imitate Wi-Fi fading and interference. Adds a fixed one-way delay plus Gaussian jitter. Like a real Wi-Fi link it keeps packets in order (FIFO). Both directions are impaired, so TCP ACKs can be lost too.
- **Media source.** A constant-bit-rate codec: one 1000-byte packet every 20 ms (50 packets/s, 400 kbit/s, similar to a standard-definition video call or RTP audio framing). Every packet carries its sequence number and generation timestamp, so one-way latency is measured exactly (all parts share one clock).
- **TCP (reliable, in-order).** Implemented on top of UDP with packet-based sequence numbers: sliding window, cumulative ACKs, RFC 6298 RTT estimation and RTO (minimum 200 ms, back-off ×2), initial window 10, slow start, congestion avoidance, fast retransmit after 3 duplicate ACKs, limited transmit (RFC 3042) and NewReno fast recovery (RFC 6582). The receiver delivers strictly in order. ACKs echo the sender timestamp, like the TCP timestamp option.
- **UDP (plain).** Sends each packet once; the receiver plays whatever arrives.
- **UDP + FEC.** Plain UDP plus one XOR parity packet per 4 data packets (25 % extra payload).

Each run lasts 20 s (1,000 media packets) and is repeated for every transport under the six conditions in Table 1, which were chosen to match published Wi-Fi and mobile measurements, and for a loss sweep from 0 % to 15 % (25 ms delay, 5 ms jitter). Loss models use a fixed random seed so all three transports see the same network. Every run writes a per-packet CSV log (sequence number, generation time, delivery time, latency); the summary metrics are calculated from these logs by experiment.py and the graphs by plots.py.

Scenario	Loss	One-way delay	Jitter (σ)	Loss pattern	Approx. RTT
Wired LAN	0 %	2 ms	0.5 ms	–	4 ms
Good Wi-Fi	1 %	15 ms	3 ms	random	30 ms
Busy campus Wi-Fi	3 %	25 ms	8 ms	random	50 ms
Congested Wi-Fi	5 %	35 ms	12 ms	random	70 ms
Weak 4G / mobile	10 %	50 ms	15 ms	random	100 ms
Bursty Wi-Fi (fading)	5 %	25 ms	8 ms	bursts, mean 3 pkts	50 ms

Table 1. Network scenarios (loss and delay apply in each direction).

Metrics. On-time % = packets that reached the receiver within 150 ms, out of all packets generated (this is what the viewer actually sees). Latency mean/median/95th/99th percentile/maximum. Jitter with the RFC 3550 RTP estimator $J = J + (|D| - J)/16$. Longest freeze = longest run of consecutive packets that were lost or late, × 20 ms. Overhead = extra bytes on the wire (headers, retransmissions, parity) relative to the payload. For TCP we also count retransmissions, RTO timeouts and fast retransmits and record the congestion window.

4. Results

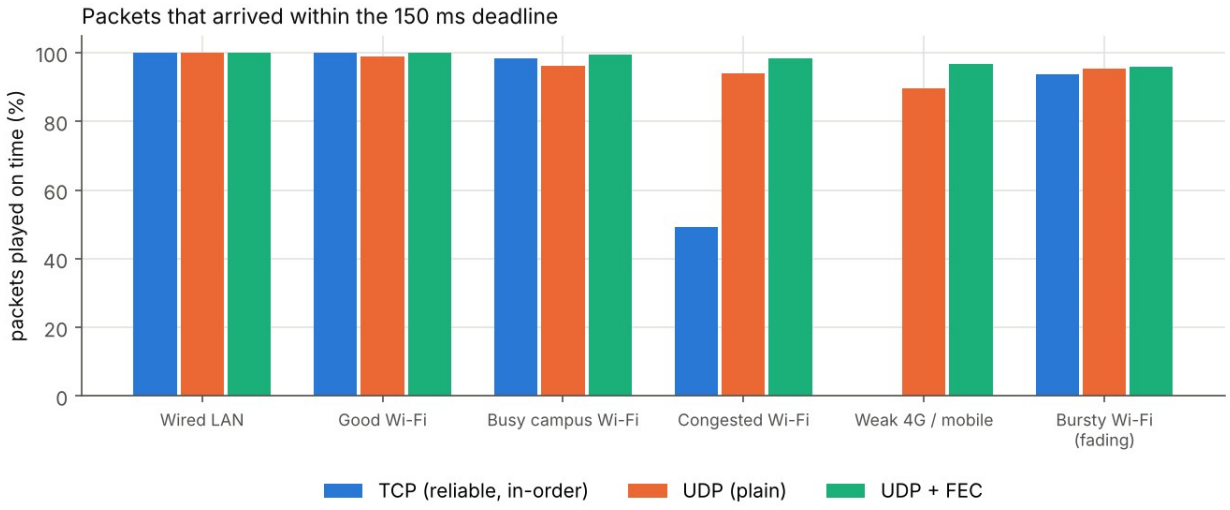
Scenario	Transport	Delivered %	On-time %	Median (ms)	p95 (ms)	p99 (ms)	Jitter (ms)	Longest freeze (ms)	Overhead %
Wired LAN	TCP	100.0	100.0	2.9	4.4	7.5	0.68	0	0.9
	UDP	100.0	100.0	2.9	4.0	5.2	0.87	0	0.9
	UDP+FEC	100.0	100.0	2.9	3.8	4.8	0.79	0	26.0
Good Wi-Fi	TCP	100.0	100.0	16.1	24.9	94.9	5.44	0	1.7
	UDP	99.0	99.0	16.1	21.2	24.9	3.58	20	0.9
	UDP+FEC	100.0	100.0	16.2	21.5	37.8	3.44	0	26.0

Busy campus Wi-Fi	TCP	100.0	98.3	29.0	118.7	161.4	11.98	100	3.0
	UDP	96.1	96.1	26.0	39.1	44.2	6.76	40	0.9
	UDP+FEC	99.5	99.5	26.2	41.0	78.4	8.77	40	26.0
Congested Wi-Fi	TCP	100.0	49.3	151.6	374.4	484.6	30.36	2180	10.8
	UDP	94.0	94.0	36.4	55.6	65.7	10.09	40	0.9
	UDP+FEC	98.4	98.4	37.9	60.0	100.3	15.66	40	26.0
Weak 4G / mobile	TCP	94.2	0.1	4611.9	6312.7	6403.6	30.06	19980	24.2
	UDP	89.5	89.5	53.0	76.1	88.0	12.63	60	0.9
	UDP+FEC	96.8	96.8	54.1	87.2	114.3	14.74	60	26.0
Bursty Wi-Fi (fading)	TCP	100.0	93.6	28.8	163.3	235.9	11.68	220	6.9
	UDP	95.4	95.4	26.2	39.7	46.3	7.06	160	0.9
	UDP+FEC	95.8	95.8	26.3	39.7	49.6	9.32	200	26.0

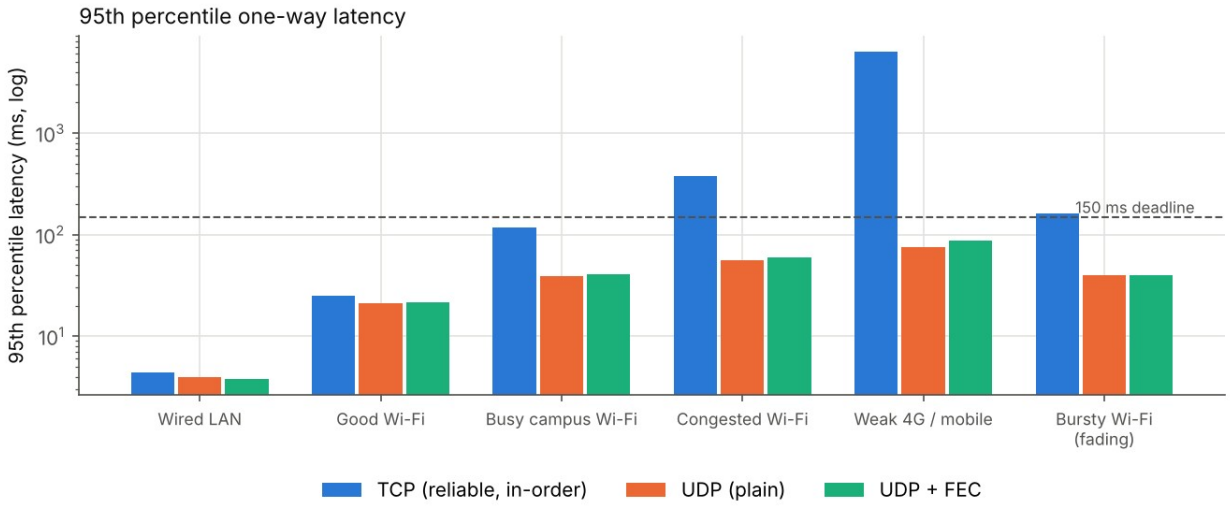
Table 2. Main results, 1,000 packets per run. Latency is one-way, from generation to delivery to the application.

Scenario	Data drops on link	ACK drops	Retransmissions	Fast retransmits	RTO timeouts	On-time goodput (kbit/s)
Wired LAN	0	0	0	0	0	400.0
Good Wi-Fi	8	10	8	8	0	400.0
Busy campus Wi-Fi	21	43	21	19	0	393.2
Congested Wi-Fi	59	61	98	42	10	197.2
Weak 4G / mobile	115	116	285	57	26	0.4
Bursty Wi-Fi (fading)	31	44	59	16	2	374.4

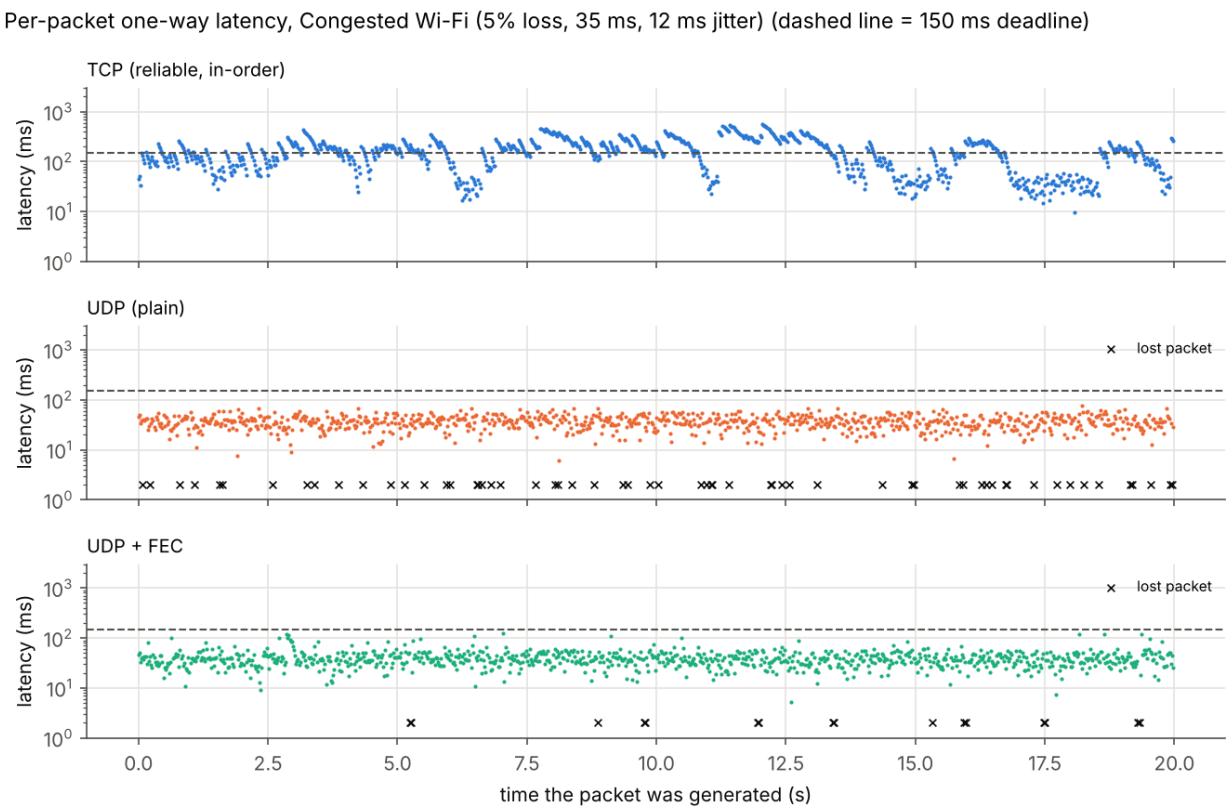
Table 3. TCP internals per scenario (target goodput is 400 kbit/s).



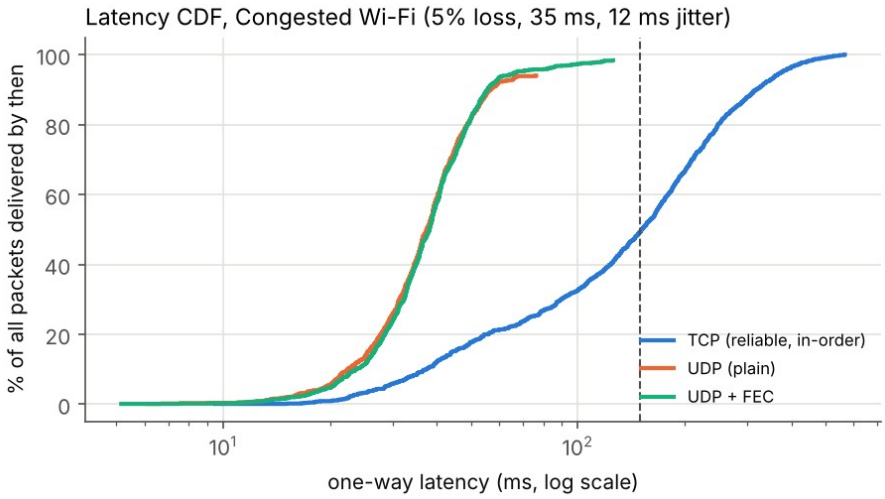
Graph 2. Share of packets that reached the receiver within the 150 ms deadline.



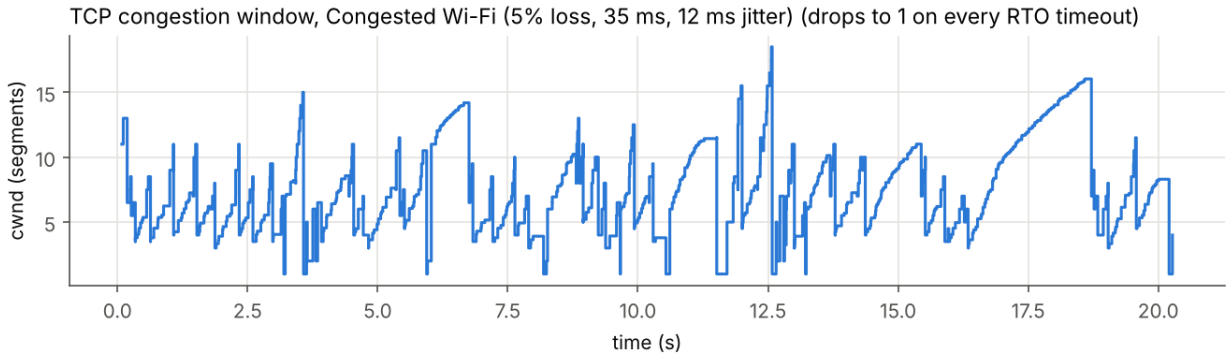
Graph 3. 95th percentile one-way latency (log scale). Dashed line: 150 ms deadline.



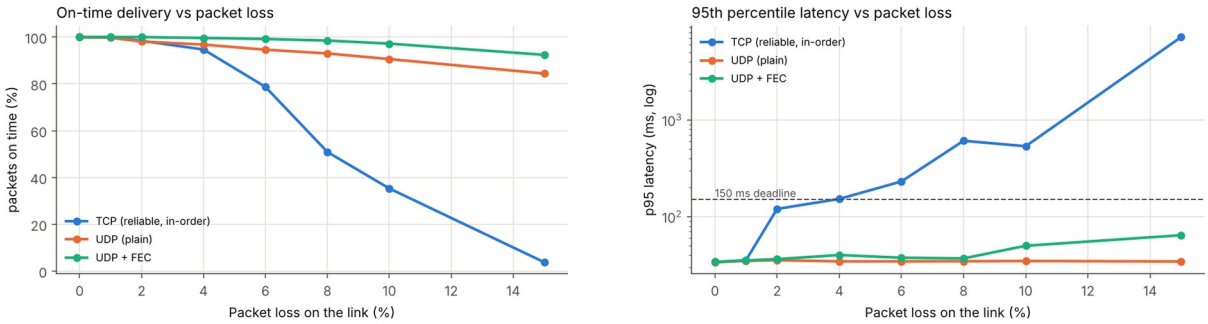
Graph 4. Latency of every packet in the Congested Wi-Fi run. Crosses mark packets that never arrived.



Graph 5. Latency CDF in Congested Wi-Fi; curves end below 100 % where packets were lost.



Graph 6. TCP congestion window in Congested Wi-Fi.



Graph 7. Loss sweep at 25 ms delay, 5 ms jitter: (a) on-time delivery, (b) 95th percentile latency.

Loss on link	TCP on-time %	TCP p95 (ms)	UDP on-time %	UDP+FEC on-time %	UDP+FEC p95 (ms)
0 %	100.0	33.8	100.0	100.0	34.1
1 %	100.0	35.5	99.7	100.0	35.3
2 %	98.4	120.1	98.1	100.0	36.4
4 %	94.7	152.3	96.8	99.6	40.1
6 %	78.7	230.9	94.6	99.2	37.6
8 %	50.9	608.2	93.0	98.5	37.0
10 %	35.3	534.3	90.6	97.2	50.0
15 %	3.8	7175.3	84.4	92.4	64.3

Table 4. Loss sweep.

5. Analysis: answers to the key questions

Q1. Which transport gives the best call on a good network? With no loss (Wired LAN) all three transports are identical: 100 % on time, median 2.9 ms. With 1 % loss (Good Wi-Fi) TCP still plays everything on time, because its retransmissions arrive within the 150 ms budget, although its p99 latency rises to 94.9 ms compared with 24.9 ms for UDP. Plain UDP loses exactly the 1 % that the link drops, and FEC repairs all of them.

Q2. What happens as loss increases? From 3 % loss onwards TCP gets worse much faster than the loss rate. In Congested Wi-Fi (5 % loss) TCP delivers 100 % of the data but only 49.3 % of it in time; its median latency is 151.6 ms and the longest freeze is 2.18 s. Plain UDP plays 94.0 % with a median of 36.4 ms and never freezes for more than 40 ms. On Weak 4G TCP collapses completely: 0.1 % on time, median 4.6 s, i.e. the call is unusable, while UDP still plays 89.5 %. The loss sweep (Graph 7, Table 4) shows the same cliff: TCP is fine up to about 2 %, drops to 78.7 % on time at 6 % loss and 3.8 % at 15 %, while UDP falls only linearly with the loss rate.

Q3. Why is TCP so much worse when it delivers more data? Three TCP mechanisms combine. (1) Head-of-line blocking: in Graph 4 every TCP loss produces a 'sawtooth' – the lost packet waits for its retransmission and all packets behind it wait too, then they are released at once with steadily falling latency. (2) Retransmission timeouts: when a loss cannot be detected by three duplicate ACKs (small window, lost retransmission or lost ACKs) TCP waits for the RTO, at least 200 ms and doubled on each repeat; there were 10 timeouts in Congested Wi-Fi and 26 on Weak 4G. A single RTO alone exceeds the whole 150 ms budget. (3) Congestion control misreads wireless loss as congestion: Graph 6 shows the window halved on every fast retransmit and reset to 1 on every timeout, although the link is not congested at all. When the window is small TCP sends less than the 50 packets/s the codec produces, so data queues in the send buffer and the delay keeps growing. The Mathis model predicts this: with RTT ≈ 100 ms and p = 0.10 on Weak 4G, $1.22/(0.1 \cdot \sqrt{0.1}) \approx 39$ packets/s, which is below the 50 packets/s the call needs, so the backlog can never drain (median latency 4.6 s, on-time goodput 0.4 kbit/s out of 400). At 5 % loss and 70 ms RTT the limit is about 78 packets/s, close enough to 50 that timeouts push TCP below it for long periods.

Q4. Does FEC fix the problem? Yes, for random loss. With one parity packet per four data packets, FEC rebuilt 44 lost packets in Congested Wi-Fi and 71 on Weak 4G, raising on-time delivery from 94.0 % to 98.4 % and from 89.5 % to 96.8 %, with only a small latency cost (p95 60.0 ms vs 55.6 ms, because a rebuilt packet must wait for the parity packet at the end of its group). In the loss sweep FEC keeps on-time delivery above 97 % up to 10 % loss. The cost is a fixed 25 % payload overhead (26.0 % on the wire) – but TCP also spent 24.2 % extra on retransmissions on Weak 4G and still failed.

Q5. What about bursty loss, which is typical of Wi-Fi? Not much. With the same 5 % average loss arriving in bursts (Bursty Wi-Fi), FEC only improves on-time delivery from 95.4 % to 95.8 %, because a burst usually removes two or more packets from one group of four, and a single XOR parity can rebuild only one. The longest freezes for UDP and FEC (160 and 200 ms) are also longer than with random loss. Real systems handle this with interleaving (spreading consecutive packets over different FEC groups), stronger codes such as Reed-Solomon, or adaptive FEC that adds more parity when bursts are detected. Interestingly TCP did relatively well here (93.6 % on time), because one burst costs it one recovery episode instead of several.

6. Recommendations

- **For application designers:** Carry live audio and video over UDP (RTP/WebRTC, or QUIC datagrams), never over plain TCP. TCP is the right choice only for the parts of a call application that must be complete: chat messages, file sharing, signalling and recordings.
- **Make UDP resilient:** Add adaptive FEC on top of UDP and interleave packets across FEC groups so burst losses are spread out. Combine with a jitter buffer and packet loss concealment, and lower the codec bit rate when loss is high instead of retransmitting.
- **For campus network administrators:** Wireless loss, not bandwidth, is what breaks calls. Reduce it with more access points at lower transmit power, 5 GHz/6 GHz bands to escape 2.4 GHz interference, and QoS (Wi-Fi WMM / DSCP EF marking) that gives voice and video priority over bulk downloads. Keep one-way delay well under 150 ms.
- **TCP fallback:** If a call must fall back to TCP (for example through a firewall that blocks UDP), expect freezes at 3 % loss and more; SACK, RACK-TLP and a smaller minimum RTO in modern stacks reduce but do not remove head-of-line blocking.

7. Limitations

The TCP is our own implementation of the standard algorithms over UDP, not the operating system's TCP, and counts segments rather than bytes; it lacks SACK and RACK-TLP, which modern Linux and Windows stacks use and which would reduce timeouts somewhat. The network is emulated on one machine, so there is no real radio, no bandwidth limit and no competing traffic; loss is independent per packet except in the bursty scenario. Each condition was run once for 20 s with a fixed seed; repeating runs with other seeds would give confidence intervals. These simplifications affect the exact numbers but not the main mechanisms, which follow directly from how TCP and UDP are defined.

8. Overall interpretation and conclusion

The experiment shows clearly why real-time applications do not use TCP for media. On a clean network all three transports behave the same. As soon as the wireless link loses a few percent of its packets, TCP's reliability turns into lateness: head-of-line blocking holds back packets that already arrived, retransmission timeouts of 200 ms or more exceed the 150 ms conversational budget on their own, and congestion control treats random wireless loss as congestion and cuts the sending rate below the rate of the call. At 5 % loss TCP delivered everything but played only 49.3 % in time, and on weak 4G the call failed completely. Plain UDP degraded gracefully, losing about as many packets as the link dropped while keeping latency near the propagation delay. UDP with simple XOR FEC gave the best result in every lossy random-loss scenario (≥ 96.8 % on time up to 10 % loss) for a fixed 25 % overhead, but needs interleaving or stronger codes to cope with bursty Wi-Fi loss. The answer to the real-world problem is therefore: run live calls over UDP with adaptive FEC, and on the network side reduce wireless loss and prioritise real-time traffic, because for interactive media a late packet is a lost packet.

References

- J. Postel, "User Datagram Protocol," RFC 768, 1980; W. Eddy (ed.), "Transmission Control Protocol," RFC 9293, 2022.
- M. Allman, V. Paxson, E. Blanton, "TCP Congestion Control," RFC 5681, 2009; RFC 6582 (NewReno), RFC 3042 (Limited Transmit).
- V. Paxson, M. Allman, J. Chu, M. Sargent, "Computing TCP's Retransmission Timer," RFC 6298, 2011.
- H. Schulzrinne et al., "RTP: A Transport Protocol for Real-Time Applications," RFC 3550, 2003.

- A. Li, "RTP Payload Format for Generic Forward Error Correction," RFC 5109, 2007; RFC 8627 (FlexFEC).
- ITU-T Recommendation G.114, "One-way transmission time," 2003.
- M. Mathis, J. Semke, J. Mahdavi, T. Ott, "The Macroscopic Behavior of the TCP Congestion Avoidance Algorithm," ACM SIGCOMM CCR, 1997.
- J. Kurose, K. Ross, Computer Networking: A Top-Down Approach, 8th ed., Pearson, 2021 (Ch. 3 Transport Layer, Ch. 9 Multimedia Networking).